



Jenkins Advance

A large blue circle with a smaller purple circle at its bottom-left edge.

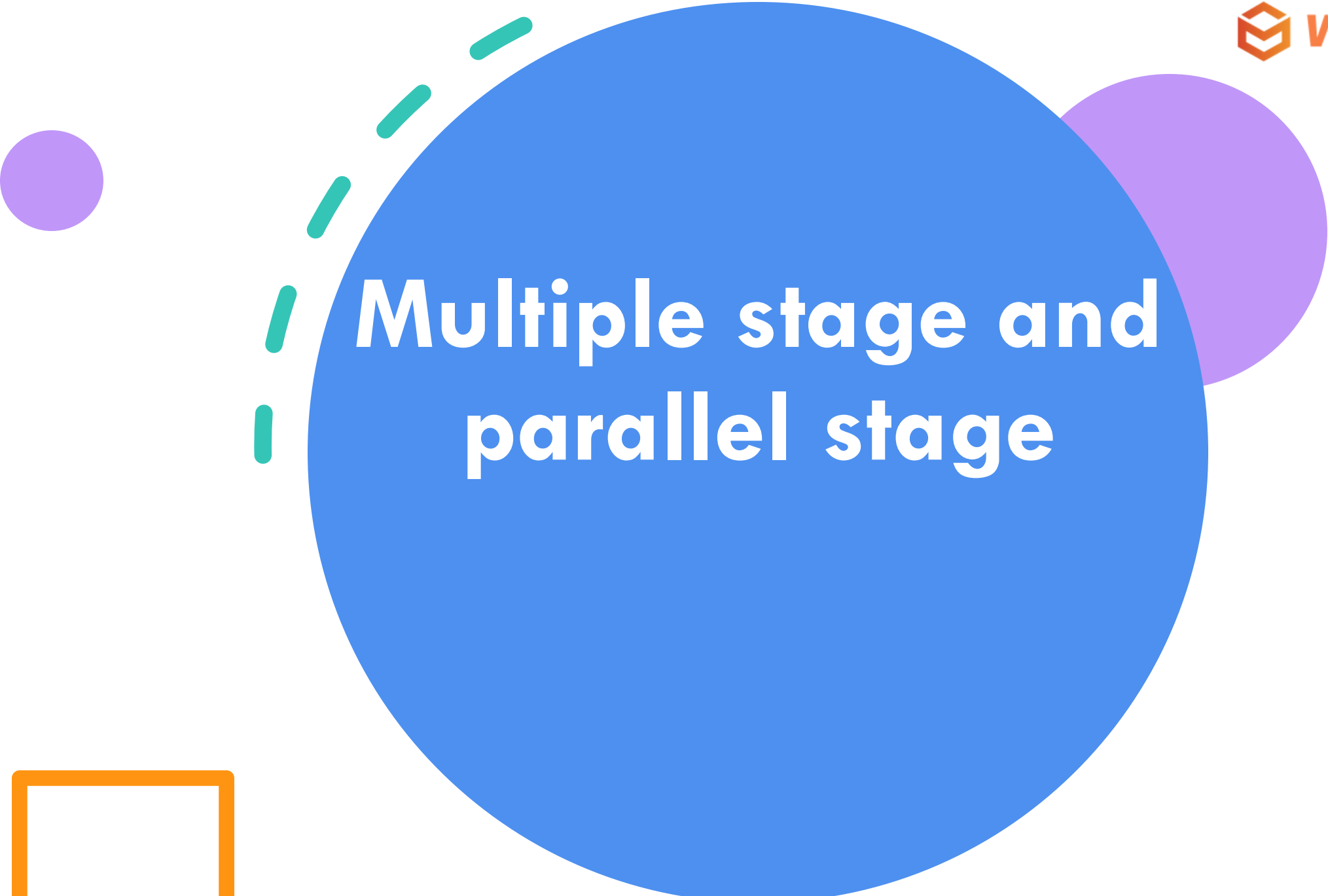
Today's Lesson

Multiple stage and parallel stage

Pipeline with parameter

Jenkins shared library

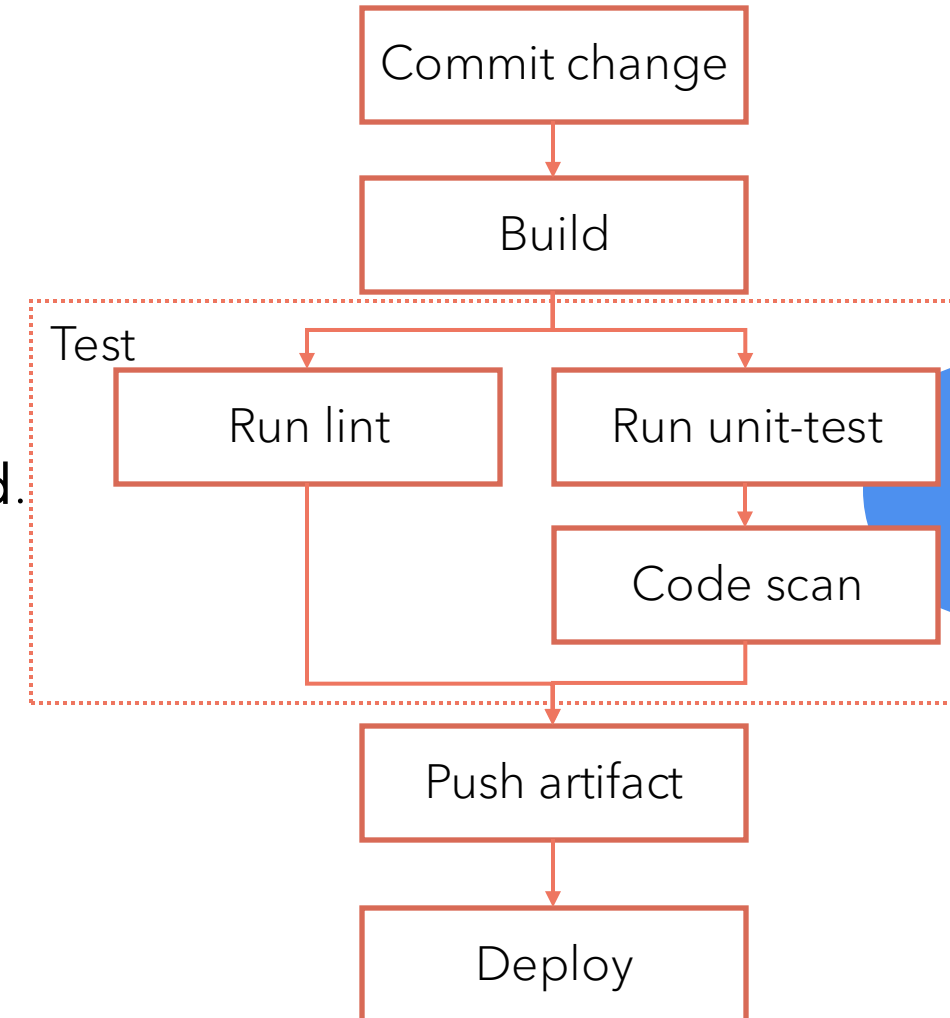
Pipeline Integration



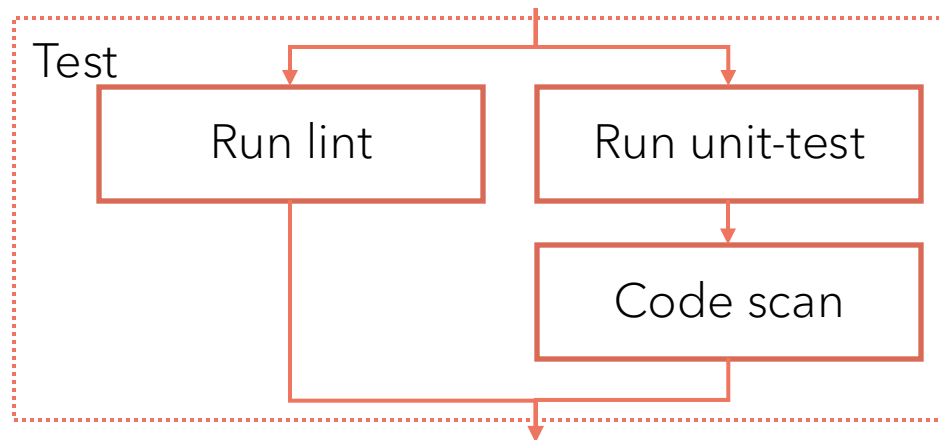
Multiple stage and parallel stage

Multiple stage and parallel stage

- Jenkins supported multiple stage and parallel stage
- Multiple-stage systems (also called *series stages*) are setups where several processing units or components are arranged **one after another**, so the **output of one stage becomes the input to the next**.
- Parallel-stage systems use two or more units that **operate at the same time**, sharing the input flow or load.



Multiple stage and parallel stage



```
stage('Test') {
  parallel(
    'Lint': {
      stage('Lint') {
        echo "Running lint checks..."
        sh "npm run lint || echo 'Lint completed'"
      }
    },
    'Unit Test & Code Scan': {
      stage('Unit Test') {
        echo "Running unit tests..."
        sh "npm test || echo 'Unit tests completed'"
      }
      stage('Code Scan') {
        echo "Running code scan..."
        sh "npm run scan || echo 'Code scan completed'"
      }
    }
  )
}
```

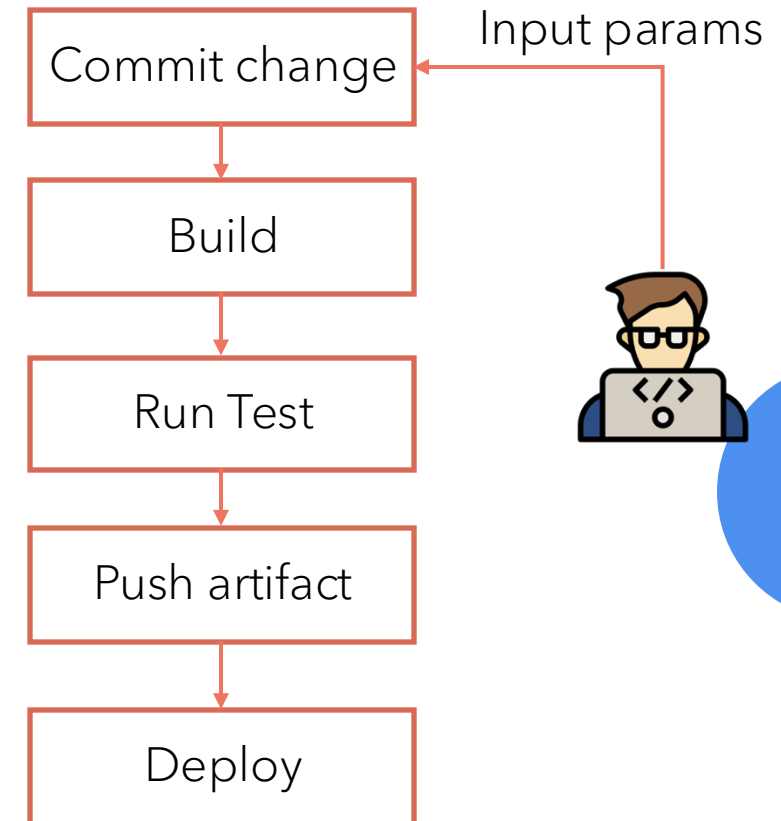
Pipeline Parameter

When a pipeline or freestyle job is configured to accept parameters, Jenkins displays input fields (like text boxes, dropdowns, checkboxes, etc.) before the build starts. The values users choose are then available to the build as environment variables.

Why Use Parameters?

Parameters are useful when you want to:

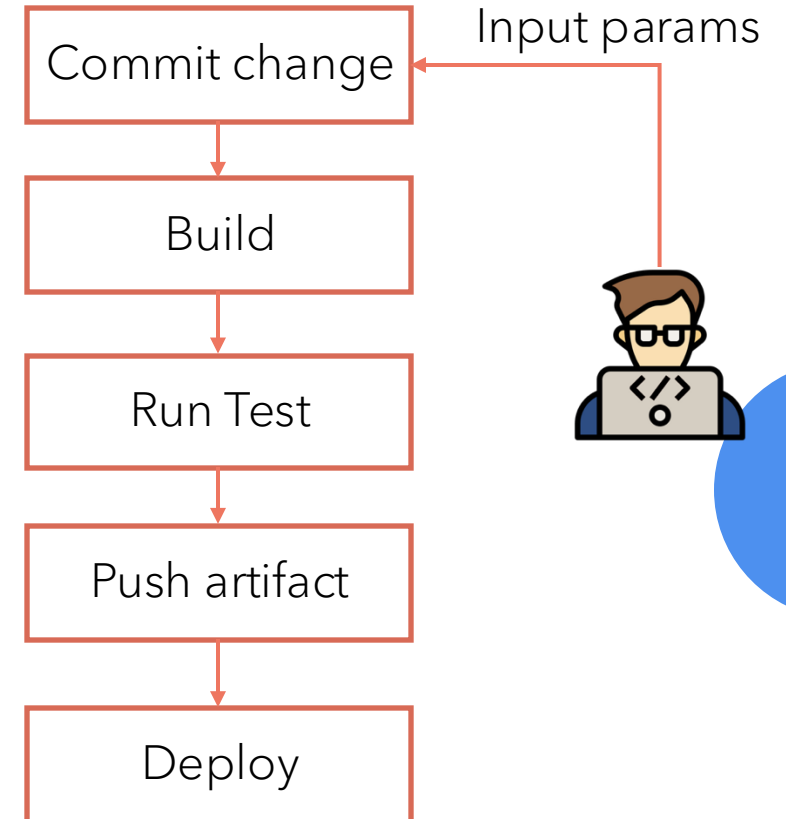
- Deploy the same pipeline to **different environments** (e.g., *dev, test, prod*).
- Build specific **branches** or **versions**.
- Pass **user input** such as configuration options, credentials, or file paths.
- **Control optional features** (e.g., run tests = true/false).



Pipeline Parameter

Example:

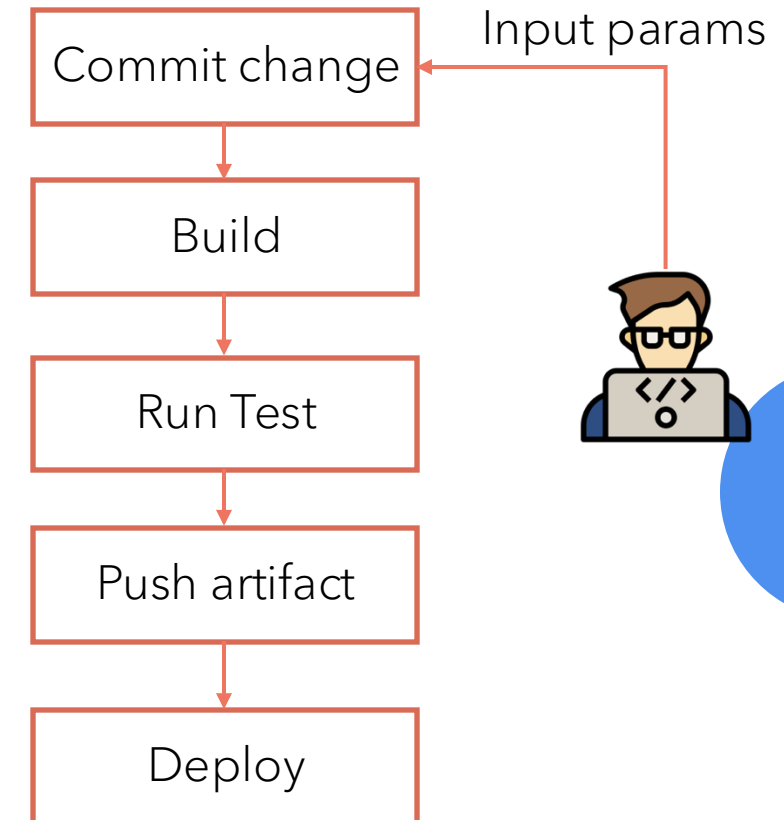
```
properties([
  parameters([
    string(name: 'ENVIRONMENT', defaultValue: 'dev', description: 'Target
environment for deployment'),
    choice(name: 'BUILD_TYPE', choices: ['debug', 'release'], description: 'Type
of build to perform'),
    booleanParam(name: 'SKIP_TESTS', defaultValue: false, description: 'Skip
running tests'),
    text(name: 'RELEASE_NOTES', defaultValue: '', description: 'Release notes for
this build')
  ])
])
```

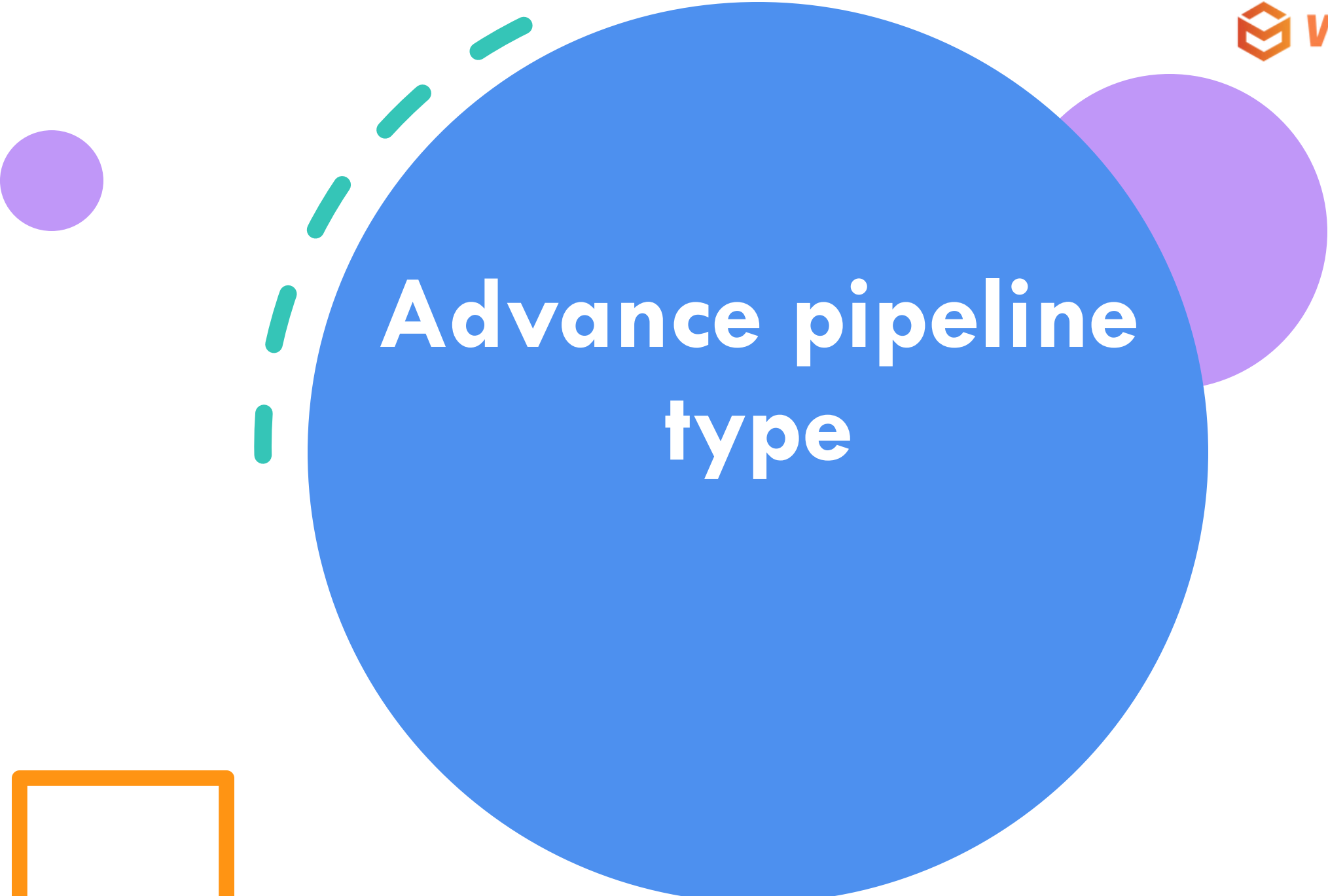


Pipeline Parameter

doc: <https://www.jenkins.io/doc/book/pipeline/syntax/>

Type	Example	Description
string	<code>string(name: 'USERNAME', defaultValue: 'admin')</code>	Single line text input
choice	<code>choice(name: 'ENV', choices: ['dev','staging','prod'])</code>	Dropdown menu
booleanParam	<code>booleanParam(name: 'RUN_TESTS', defaultValue: true)</code>	Checkbox (true/false)
password	<code>password(name: 'SECRET_KEY')</code>	Hidden input for sensitive data
text	<code>text(name: 'DESCRIPTION', defaultValue: 'Enter details')</code>	Multi-line text
file	<code>file(name: 'CONFIG_FILE')</code>	Lets users upload a file
credentials	<code>credentials(name: 'AWS_CREDENTIALS_ID')</code>	Pulls stored Jenkins credentials

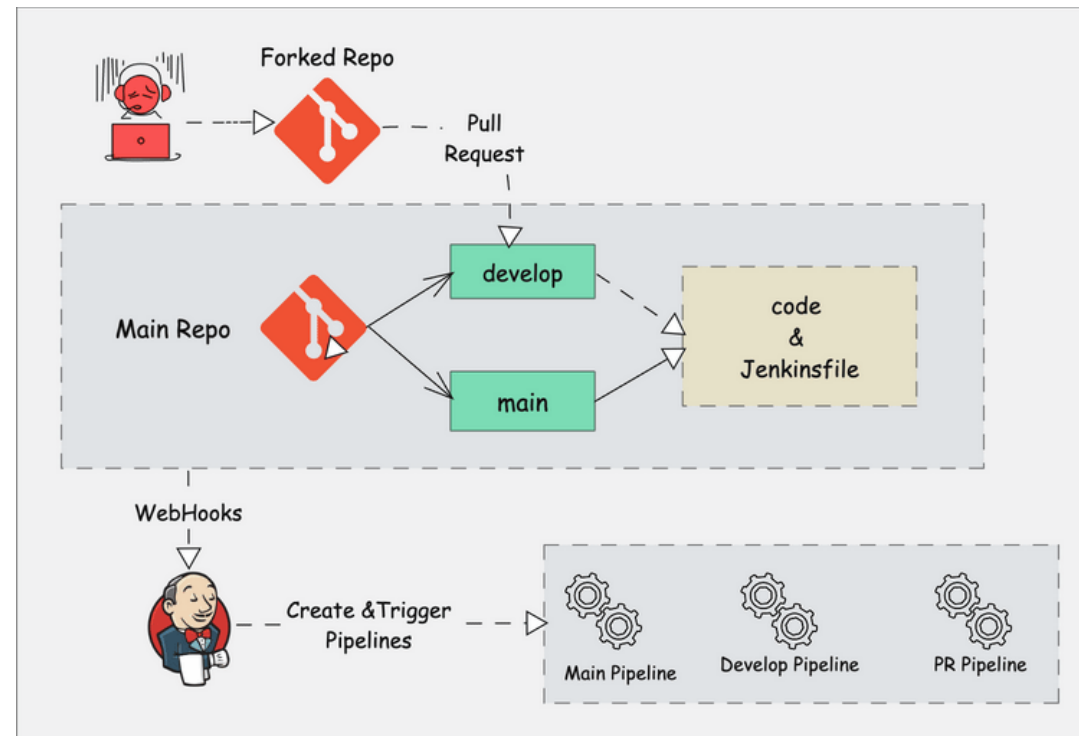




Advance pipeline type

Multiple branch pipeline

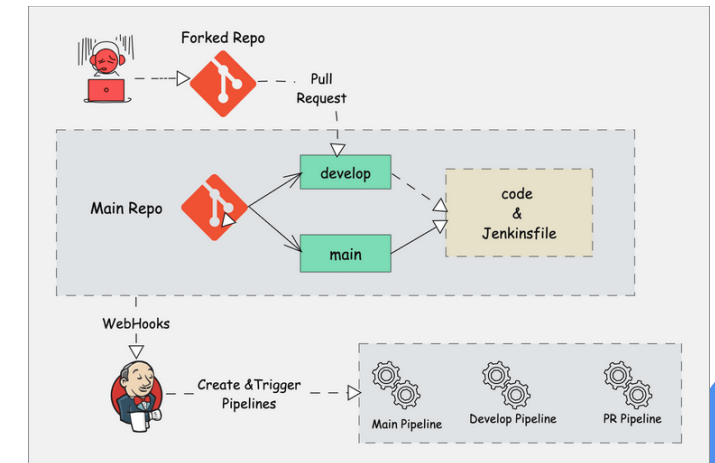
A Jenkins Multi-branch Pipeline is a specialized project type that automatically discovers, manages, and executes Pipelines for branches within a single source code repository. Instead of creating individual Jenkins jobs for each branch, a Multi-branch Pipeline dynamically detects branches containing a **Jenkinsfile** and sets up corresponding pipeline jobs.



Multiple branch pipeline

Key Features and Benefits:

- **Automatic Branch Discovery:**
 - Jenkins automatically scans a specified source code management (SCM) system (e.g., Git, GitHub, Bitbucket) and identifies branches that contain a Jenkinsfile.
- **Dynamic Pipeline Creation:**
 - For each discovered branch, a separate pipeline job is automatically created and configured based on the Jenkinsfile present in that branch.
- **Branch-Specific Workflows:**
 - Each branch can have its own Jenkinsfile, allowing for tailored build, test, and deployment processes for different development stages (e.g., feature branches, development, staging, production).
- **Pull Request Integration:**
 - Many SCM integrations support automatic discovery and pipeline execution for pull requests, enabling CI/CD workflows for proposed changes.
- **Reduced Manual Configuration:**
 - Eliminates the need to manually create and maintain separate Jenkins jobs for every new branch or feature.
- **Consistency and Version Control:**
 - The pipeline definition (Jenkinsfile) is stored alongside the application code in the SCM, ensuring that the pipeline evolves with the code.

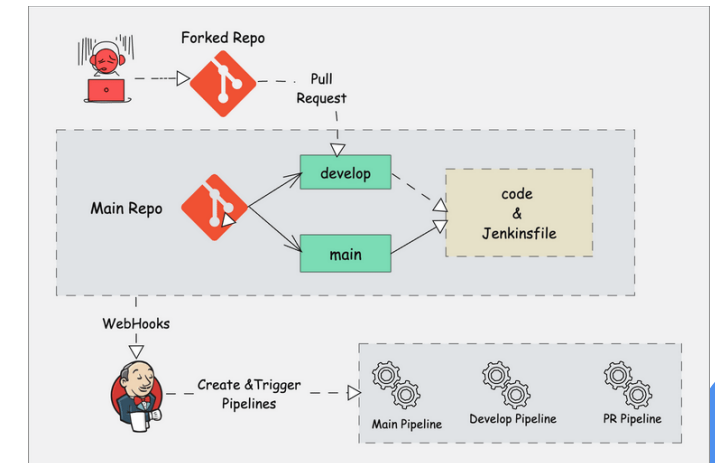


Organization Folder

An **Organization Folder** is a special Jenkins item type designed to automatically manage and build **multiple repositories** belonging to a **source control organization** (like a GitHub Organization or Bitbucket Team).

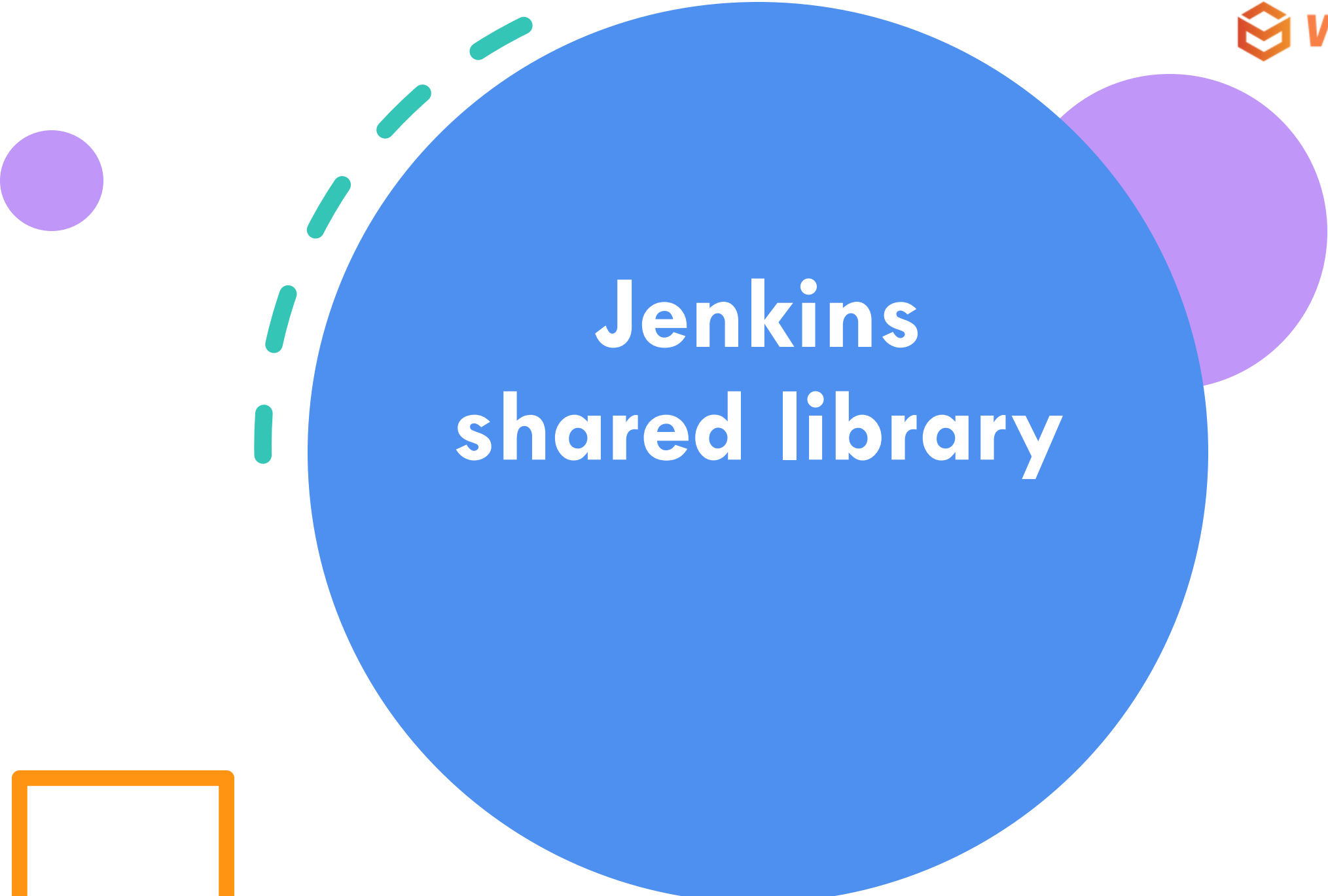
Instead of creating one pipeline job per repository by hand, an organization folder scans your whole organization or team, detects repositories that contain a **Jenkinsfile**, and automatically creates/manage pipelines for them.

In short: **Organization Folder** contains multiple "multiple branch pipeline"



Summary

Feature	Multibranch Pipeline	Organization Folder
Scope	Single repository	Multiple repositories
Discovers	Branches & PRs within one repo	Repositories within an organization, and their branches
Automation Level	Auto-manages branches	Auto-manages repos and their branches
Needs Jenkinsfile	In each branch	In each repo (per branch)
Typical Use Case	CI/CD for one project with multiple branches	CI/CD for many projects in an organization
Example	Builds main , develop , feature1 of repo1	Scans repo1, repo2, repo3 – and runs multibranch jobs for each



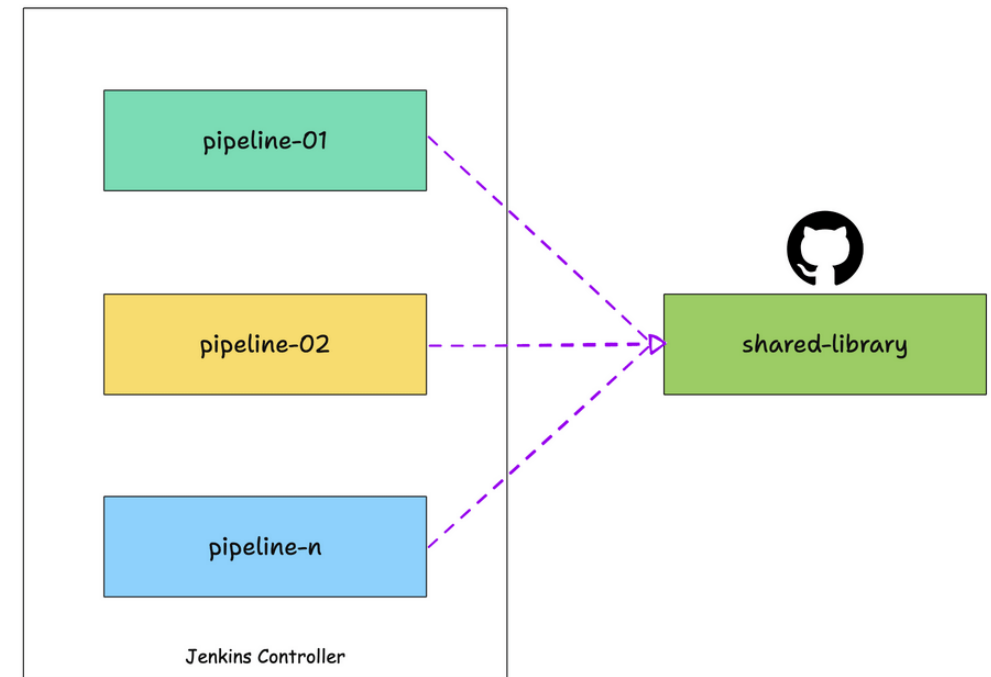
Jenkins shared library

Jenkins shared library

A Jenkins Shared Library is a reusable collection of **pipeline code** (written in Groovy) that can be shared across multiple Jenkins pipelines.

Instead of repeating the same pipeline stages or scripts in every Jenkinsfile, you store common logic once in a shared library, and all pipelines can call it.

Think of it as a **central library of reusable pipeline functions, steps, or stages** – like a helper or framework layer for Jenkins pipelines.



Jenkins shared library

Why Use a Shared Library?

Without shared libraries:

Each Jenkinsfile contains a lot of repeated code (build, test, deploy, notify, etc.).

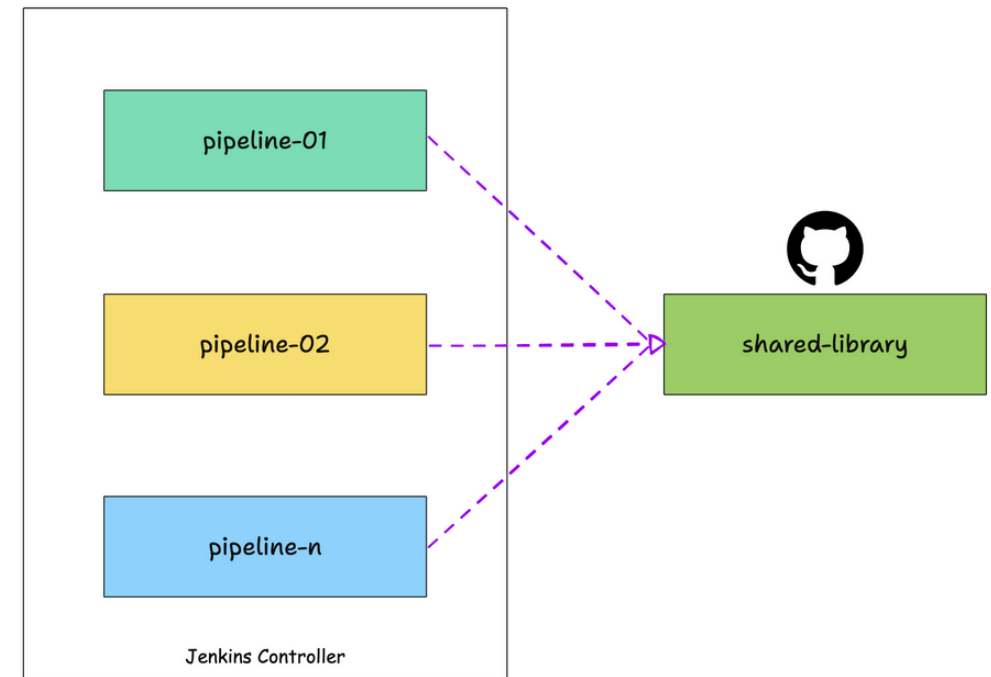
Updating logic across many pipelines is hard.

With shared libraries:

Common pipeline logic lives in one place.

Pipelines just import and reuse it.

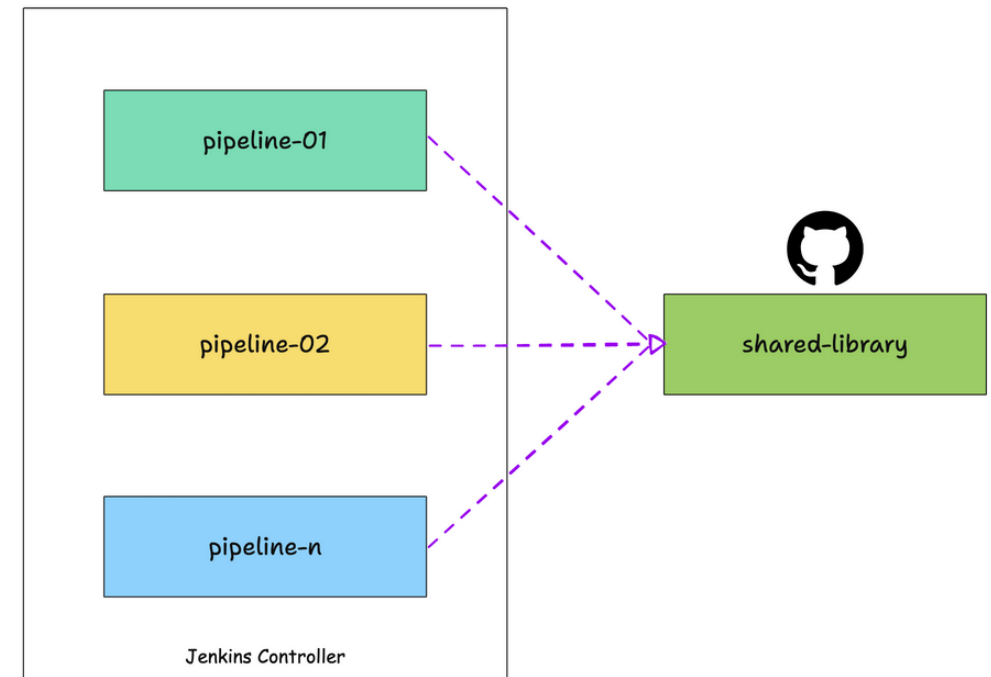
Easier to maintain, version, and standardize processes (e.g., build conventions, deployment methods, notifications).



Jenkins shared library

How a Shared Library Works

- You define shared code in a **Git repository** (or folder).
- In Jenkins, you configure that repo as a **Global Shared Library** (or load it from within your Jenkinsfile).
- Pipelines load the library and call its functions.



File Structure

Folder meanings:

- **vars/** → Simple scripts exposed as global pipeline steps (callable by name).
- **src/** → Regular Groovy classes, organized by package name (for complex logic).
- **resources/** → Files accessible in the library (e.g., templates, YAML configs).

```
(root)
├── vars/
│   ├── buildApp.groovy
│   └── deployApp.groovy
├── src/
│   └── org/example/util/Helper.groovy
├── resources/
│   └── config.yml
└── README.md
```

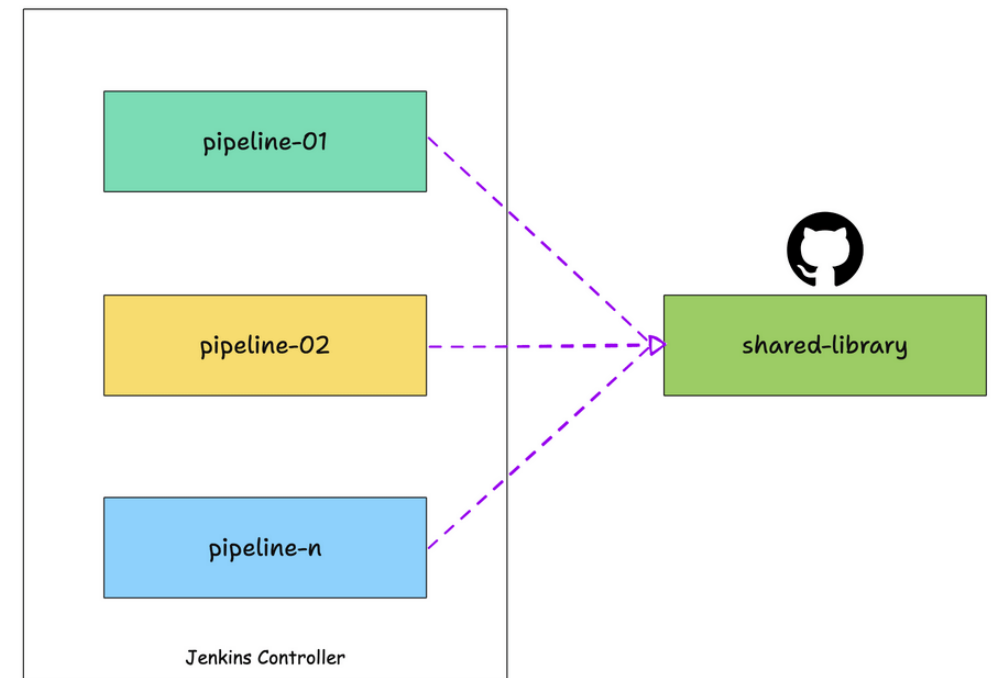
Usage

In shared library repo: `vars/deployApp.groovy`

```
def call() {  
  node {  
    stage('Build') {  
      echo "Building the project..."  
    }  
    stage('Test') {  
      echo "Running tests..."  
    }  
    stage('Deploy') {  
      echo "Deploying application..."  
    }  
  }  
}
```

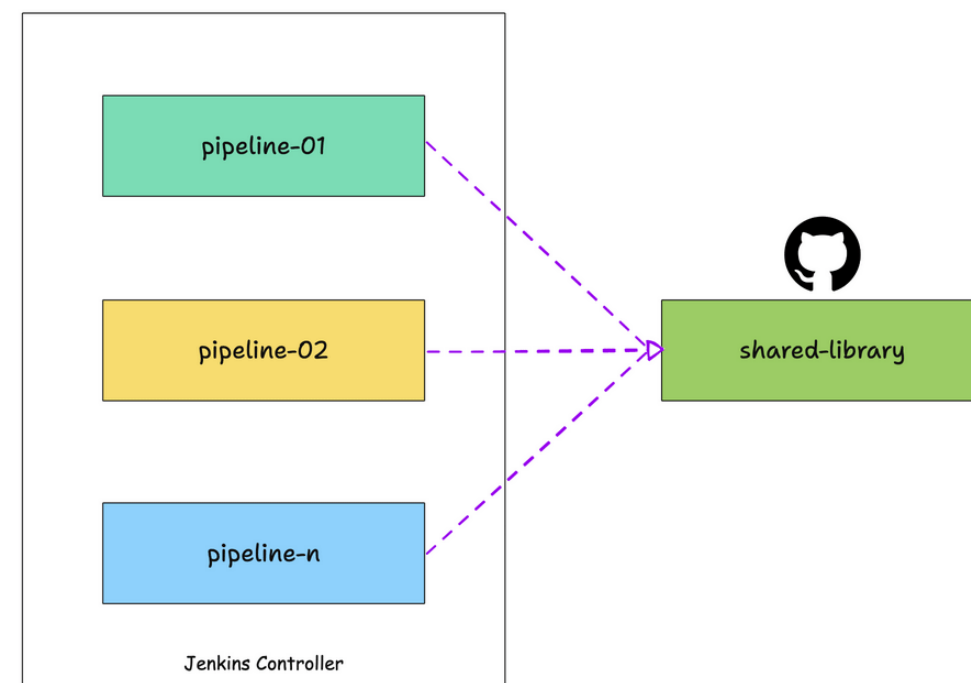
In app repo Jenkinsfile:

```
@Library('my-shared-library') _ // Loads the shared library (previously configured  
in Jenkins)  
  
deployApp()
```



Key Benefits

Benefit	Description
Reusability	Share common build, deploy, or test logic across many pipelines.
Maintainability	Change code once – all pipelines benefit.
Modularity	Organize code cleanly using Groovy classes and functions.
Version Control	Tag or branch shared libraries for stable versions.
Consistency	Enforces standard CI/CD practices across projects.



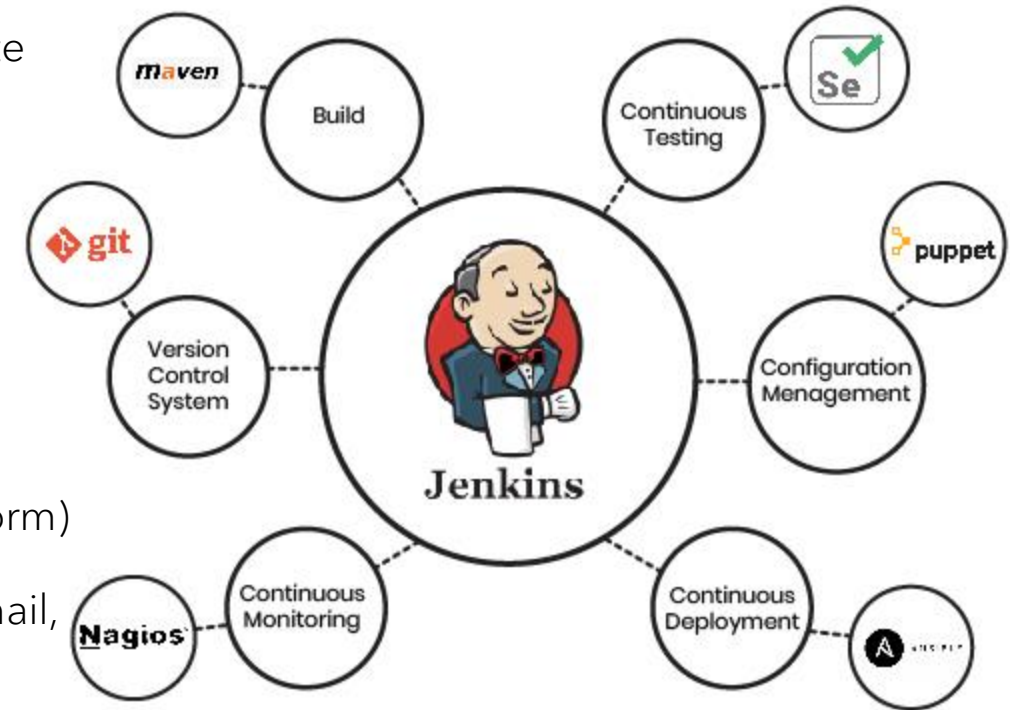


Bonus Integration with 3rd party

Integration with 3rd party

Integration with third-party tools means Jenkins can communicate with, trigger, or be triggered by other software – such as:

- Source control tools (GitHub, GitLab, Bitbucket)
- Build and artifact tools (Maven, Gradle, npm, Docker, Nexus, Artifactory)
- Testing frameworks (JUnit, Selenium, SonarQube)
- Cloud & infrastructure (AWS, Azure, GCP, Kubernetes, Terraform)
- Notification & collaboration tools (Slack, Microsoft Teams, email, Jira)
- Security/DevOps tools (HashiCorp Vault, Snyk, Anchore)

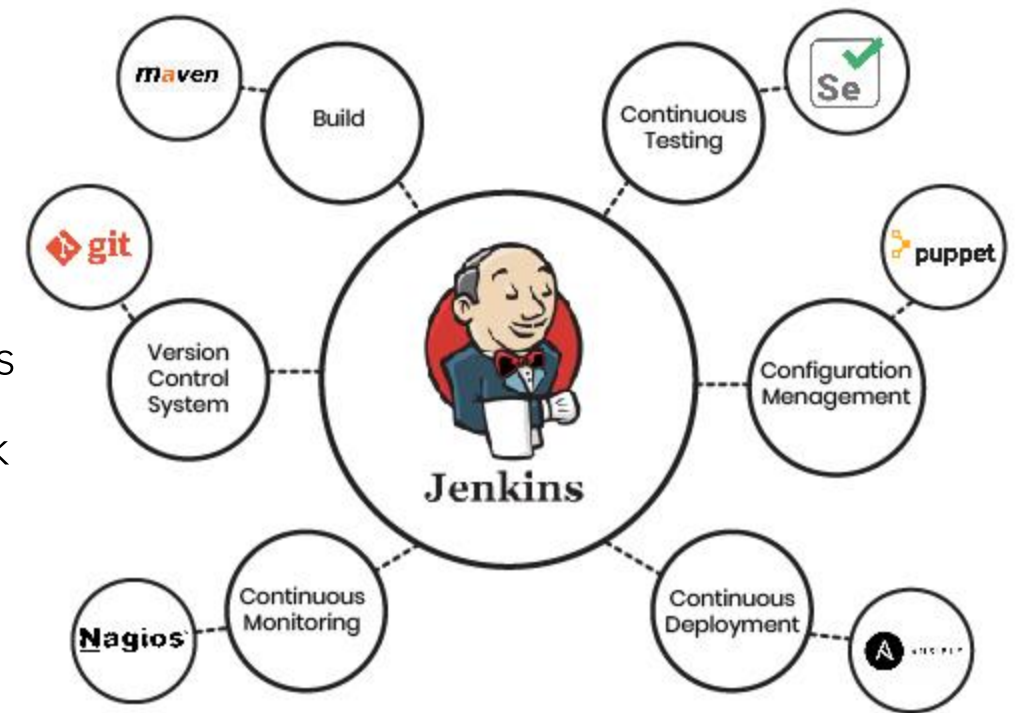


Using Plugins

Jenkins has **1000+ plugins** that extend its abilities to interact with external systems.

Examples:

- **Git plugin** → Integrates with Git repositories
- **Docker plugin** → Builds and runs Docker containers
- **Slack plugin** → Sends pipeline notifications to Slack
- **SonarQube plugin** → Runs code quality analysis
- **AWS / Kubernetes plugins** → Deploys to cloud environments
- Each plugin adds pipeline steps or post-build actions that make integration seamless.



Using pipeline logic

You can integrate directly inside the **Jenkinsfile** using shell commands or tool-specific CLIs.

This pipeline integrates with:

- **Maven** for build
- **SonarQube** for code analysis
- **Docker** for containerization
- **Slack** for notifications

```
node { // 'node' defines the executor / agent
  try {
    stage('Build') {
      echo "Starting build..."
      sh 'mvn clean package'
    }

    stage('Static Code Analysis') {
      echo "Running SonarQube analysis..."
      sh 'sonar-scanner'
    }

    stage('Docker Build & Push') {
      echo "Building Docker image..."
      sh 'docker build -t myimage:latest .'

      echo "Pushing Docker image to registry..."
      sh 'docker push myregistry/myimage:latest'
    }

    stage('Notify') {
      echo "Sending notification to Slack..."
      slackSend(channel: '#builds', message: "Build completed successfully")
    }
  } catch (err) {
    // Handle failure and send alert to Slack
    echo "Build failed: ${err}"
    slackSend(channel: '#builds', message: "Build failed: ${err}")
    // Make sure Jenkins marks the build as failed
    currentBuild.result = 'FAILURE'
    throw err
  } finally {
    echo "Pipeline finished."
  }
}
```




Practice Lab